

Trabajo fin de ciclo

Desarrollo de aplicaciones Multiplataforma (DAM)

MERCADILLOS EN CASA (MEC)

ÍNDICE:

1. Motivación e introducción.....	3
1.1 Introducción.....	3
1.2 Motivación.....	3
1.3 Contexto y problema a resolver.....	3
1.4 Objetivos propuestos.....	4
2. Metodología, recursos y planificación.....	4
2.1 Metodología utilizada.....	4
2.2 Estimación de recursos.....	5
2.3 Planificación (Hitos y cronograma).....	5
3. Tecnologías y herramientas.....	6
4. Análisis.....	7
4.1 Definición de requisitos.....	7
4.2 Modelo de datos (MariaDB).....	8
4.3 Casos de uso.....	9
5. Diseño.....	10
5.1 Diseño general / arquitectura.....	10
5.2 Diagrama de clases (UML).....	11
5.3 Mockups / capturas de interfaz y navegación.....	11
5.4 Decisiones de diseño importantes.....	12
6. Implementación.....	12
6.1 Estructura del proyecto.....	12
6.2 Componentes principales.....	13
6.3 Dificultades encontradas y soluciones.....	14
7. Multiusuario concurrente.....	15
7.1 Identificación y gestión de sesiones.....	15
7.2 Concurrencia de usuarios simultáneos.....	15
8. Despliegue e instalación.....	17
8.1 Manual de instalación.....	17
8.2 Configuración.....	18
8.3 Datos de ejemplo.....	18
8.4 Manual de usuario.....	19
9. Pruebas.....	20
9.1 Plan de pruebas.....	20
10. Resultados, conclusiones y vías futuras.....	21
10.1 Objetivos alcanzados.....	21
11. Bibliografía / Webgrafía.....	23

1. Motivación e introducción

1.1 Introducción

Mercadillos en Casa (MEC) es una aplicación responsive diseñada para digitalizar y centralizar el sector de los eventos de venta temporales, tales como vaciados de casas, rastros solidarios y mercados vintage en España. El proyecto está dirigido a dos perfiles principales: los organizadores que necesitan una plataforma para dar visibilidad a sus eventos, y los visitantes interesados.

El objetivo principal de MEC es resolver el problema de la fragmentación de la información en el sector. Actualmente, los anuncios de eventos se diluyen entre el contenido de redes sociales genéricas, complicando su descubrimiento. Esta aplicación aborda este problema centralizando todos los datos mediante un sistema de gestión y un mapa interactivo, ofreciendo una solución más organizada y accesible.

Las funcionalidades clave incluyen:

1. Geolocalización en tiempo real: Visualización de eventos en un mapa dinámico con marcadores.
2. Autogestión de eventos (CRUD): Los organizadores pueden crear, editar, eliminar y añadir galerías de imágenes a sus anuncios.
3. Sistema de filtros avanzado: Búsqueda segmentada por categorías y fechas.
4. Sistema de usuarios y favoritos: Gestión de sesiones y capacidad para que los visitantes guarden eventos de interés.

1.2 Motivación

La motivación principal de este proyecto es profesional y personal. Surge de la necesidad real de "Mercadillos en Casa", una empresa emergente (startup) que busca digitalizar el sector de los eventos efímeros y sostenibles.

A partir de la detección de un nicho de mercado vacío en España, la falta de una plataforma tecnológica que centralice vaciados de casas y mercadillos sostenibles, el objetivo tecnológico de este proyecto es transformar dicha visión de negocio en una herramienta funcional, escalable y profesional. Para ello, se asume el rol de responsable de tecnología con el fin de ejecutar el lanzamiento del Producto Mínimo Viable (MVP).

1.3 Contexto y problema a resolver

Contexto inicial: La empresa actualmente se apoya en métodos tradicionales de comunicación y redes sociales genéricas que no permiten la geolocalización precisa ni la gestión eficiente de eventos por parte de los organizadores. Existe una dependencia de procesos manuales para informar a los clientes sobre la ubicación y horarios de los mercadillos.

La Solución (MEC): Este proyecto ofrece una aplicación responsive. Esta solución aporta automatización: los organizadores pueden autogestionar sus eventos y los visitantes pueden encontrarlos en un mapa dinámico. Esto profesionaliza la imagen de la marca y permite que el negocio escale a nivel nacional.

1.4 Objetivos propuestos

Objetivos generales:

1. Desarrollar una aplicación responsive funcional que centralice la oferta de mercadillos temporales y eventos de venta sostenible en España.
2. Facilitar el consumo local y de proximidad conectando a organizadores y visitantes mediante un sistema de geolocalización.

Objetivos Específicos:

1. Diseñar e implementar una base de datos relacional (MariaDB) robusta para gestionar perfiles, categorías, eventos e imágenes.
2. Crear una interfaz de usuario interactiva y fluida utilizando Vue 3, garantizando una experiencia óptima en dispositivos móviles y de escritorio.
3. Desarrollar un sistema Backend en Python (Flask) mediante una API RESTful estructurada con el patrón DTO para estandarizar las respuestas.
4. Implementar un mapa interactivo (Leaflet.js) que permita la visualización de eventos mediante marcadores visuales diferenciados.
5. Construir un sistema de gestión (CRUD) completo con capacidad para la subida de múltiples imágenes por evento.
6. Desarrollar un sistema de favoritos para mejorar la retención y experiencia de los usuarios visitantes.

2. Metodología, recursos y planificación

2.1 Metodología utilizada

Para este proyecto se ha utilizado una metodología de Desarrollo Incremental. En lugar de intentar programar todo el sistema de una sola vez, la aplicación se ha ido construyendo paso a paso.

El trabajo se ha organizado de la siguiente manera:

1. **Base del proyecto:** Primero se desarrolló lo esencial, como el registro de usuarios y la publicación de mercadillos solo con texto.
2. **Nuevas funcionalidades:** Sobre esa base estable, se fueron añadiendo las partes más difíciles una a una. Primero el mapa interactivo, luego el cálculo de coordenadas para las direcciones, y por último la subida de múltiples imágenes.

3. **Pruebas constantes:** Cada vez que se añadía algo nuevo, se probaba a fondo antes de pasar a la siguiente tarea.

Esta forma de trabajar ha sido muy útil para evitar bloqueos, ya que si algo fallaba, era fácil localizar el error sin romper el código que ya funcionaba.

2.2 Estimación de recursos

Hardware:

- **Desarrollo:** Ordenador portátil (Windows) para la codificación del Backend y Frontend
- **Pruebas:** Navegador web del ordenador y la herramienta de simulación para móviles que viene dentro de Google Chrome. Esto me permitió comprobar desde el mismo ordenador cómo se adapta el diseño de la app tanto en pantallas grandes de escritorio como en diferentes smartphones (Android e iPhone), simulando también el comportamiento de los gestos táctiles.
- **Servidor:** Inicialmente un servidor local (localhost) para desarrollo

Software:

- **IDE:** Visual Studio Code.
- **Entorno de ejecución (Backend):** Python 3.
- **Entorno de ejecución (Frontend):** Node.js.
- **Base de Datos:** MariaDB.
- **Control de versiones:** Git y GitHub.
- **Pruebas de API:** Postman.
- **Navegadores:** Google Chrome (DevTools) para depuración.

2.3 Planificación (Hitos y cronograma)

El proyecto se divide en 4 grandes hitos:

1. **Hito 1 (marzo):** Entrega de Planificación Inicial y diseño de la BD.
2. **Hito 2 (abril):** Backend funcional (API lista) y gestión de usuarios.
3. **Hito 3 (mayo):** Frontend maquetado e integración del mapa interactivo.
4. **Hito 4 (junio):** Pruebas finales, corrección de errores y entrega de memoria.

3. Tecnologías y herramientas

Lenguajes:

- **JavaScript (ES6+)**: Utilizado en el desarrollo del cliente por su dinamismo, ecosistema maduro y capacidad de computar lógica reactiva de manera eficiente en el navegador.
- **Python (v3.11)**: Elegido para el backend debido a su sintaxis limpia, alta legibilidad y la robustez que ofrece a la hora de manipular datos y gestionar peticiones del servidor.
- **SQL**: Empleado como lenguaje estándar de consulta para garantizar interacciones precisas, seguras y optimizadas con el sistema de almacenamiento de datos.
- **HTML5 / CSS3**: Utilizados para estructurar el contenido de la plataforma y aplicar un diseño responsivo, limpio y adaptado nativamente a dispositivos móviles y de escritorio.

Frameworks y Librerías:

- **Vue 3 - Composition API (v3.4)**: Seleccionado por su arquitectura reactiva basada en componentes modulares, facilitando un desarrollo ágil y un rendimiento óptimo del frontend.
- **Pinia (v2.1)**: Implementado como almacén de estado global por su ligereza, tipado nativo y perfecta sincronización con el paradigma reactivo de Vue 3.
- **Vue Router (v4.3)**: Elegido para gestionar la navegación de la Single Page Application (SPA), permitiendo transiciones fluidas entre pantallas sin recargar la página.
- **Leaflet.js (v1.9)**: Incorporado por ser una librería móvil-fuerte de código abierto extremadamente ligera, ideal para renderizar mapas interactivos de forma fluida.
- **Flask (v3.0)**: Microframework de Python escogido por su flexibilidad y minimalismo, permitiendo levantar una API REST robusta sin la sobrecarga de configuraciones innecesarias.

Base de datos:

- **MariaDB (v11.4)**: Sistema de gestión de bases de datos relacionales seleccionado por su alto rendimiento, seguridad y compatibilidad total con SQL, ideal para garantizar la integridad de las entidades del proyecto.

APIs externas:

- **Nominatim - OpenStreetMap API**: Integrada como solución gratuita y sin restricciones severas de licencia para resolver la geocodificación inversa,

transformando direcciones textuales en coordenadas precisas de forma automatizada.

Herramientas de desarrollo:

- **Visual Studio Code (v1.90):** Entorno de desarrollo integrado (IDE) estándar debido a su ligereza, versatilidad y amplio catálogo de extensiones que optimizan el flujo de trabajo.
- **Git (v2.45) & GitHub:** Utilizados para el control de versiones y el respaldo del código en la nube, asegurando un desarrollo organizado y un histórico de cambios seguro.
- **Postman (v10.24):** Herramienta clave para el testeo de la API REST, permitiendo simular y validar las peticiones del frontend antes de su integración final en el código.
- **NPM (v10.8):** Gestor de paquetes oficial de Node.js empleado para descargar, administrar y mantener actualizadas de manera automatizada todas las dependencias del frontend.
- **Vite (v5.2):** Herramienta de construcción (*bundler*) de última generación elegida por su velocidad extrema en el servidor de desarrollo y su eficiencia al compilar el proyecto para producción.

4. Análisis

4.1 Definición de requisitos

Para el desarrollo de Mercadillos en Casa (MEC), se han establecido los siguientes requisitos fundamentales:

Requisitos Funcionales (RF):

- **RF1 - Gestión de usuarios y perfiles:** El sistema debe permitir el registro y autenticación de usuarios. Además, debe permitir la edición del perfil, incluyendo la subida y actualización de una foto de perfil.
- **RF2 - Gestión de eventos (CRUD):** Los organizadores deben poder publicar, modificar y eliminar sus eventos. En la creación, el sistema debe permitir subir múltiples fotos del mercadillo. Asimismo, el servidor debe borrar de forma automática cualquier evento cuya fecha programada sea anterior al día de hoy para mantener el mapa limpio de eventos caducados.
- **RF3 - Geocodificación y Mapa interactivo:** Al introducir una dirección en texto, el sistema debe calcular automáticamente sus coordenadas (latitud y longitud) para posicionar el evento en un mapa interactivo.
- **RF4 - Búsqueda y Filtros:** El sistema debe disponer de un buscador para que los usuarios puedan filtrar los eventos visibles en el mapa por su categoría (Vintage, Vaciado, Solidario, etc.), por rangos de fechas (Desde / Hasta) y por cercanía al usuario mediante su geolocalización.

- **RF5 - Sistema de Favoritos:** Los usuarios registrados deben poder marcar eventos como favoritos para guardarlos en una lista personalizada y consultarlos más tarde.

Requisitos No Funcionales (RNF):

1. **RNF1 - Usabilidad:** La interfaz debe ser intuitiva y seguir la línea visual de la marca (limpia y amigable).
2. **RNF2 - Diseño Adaptable (Responsive Design):** La aplicación debe contar con un diseño fluido que se adapte al 100% del tamaño de la pantalla, reestructurando sus componentes dinámicamente para ser completamente accesible desde cualquier navegador móvil o de escritorio.
3. **RNF3 - Estandarización de Respuestas:** Toda comunicación entre el Frontend y el Backend debe seguir un formato estandarizado (Data Transfer Object - DTO) para asegurar que los errores y confirmaciones se procesen uniformemente.
4. **RNF4 - Seguridad:** Las contraseñas de los usuarios deben estar encriptadas en la base de datos.

4.2 Modelo de datos (MariaDB)

Para la persistencia de datos se ha seleccionado MariaDB, un sistema de gestión de bases de datos relacional.

La elección de una base de datos relacional encaja perfectamente con la naturaleza del proyecto MEC. La información está fuertemente estructurada y conectada entre sí: un usuario organiza múltiples eventos, un evento pertenece a una única categoría, y muchos usuarios pueden guardar muchos eventos como favoritos. MariaDB garantiza la integridad referencial de estos datos mediante claves foráneas y maneja de forma altamente eficiente los tipos de datos decimales necesarios para las coordenadas (latitud y longitud) del mapa.

Anexo 1. Diagrama entidad-relación

Entidades principales:

- **users (Usuarios):** user_id, users_name, email, password, user_type, phone, photo_url, registry_date, verified_email y token_verified.
- **events (Eventos):** event_id, organizer_id, category_id, title, description, address, latitude, longitude, start_date, end_date y photo_url.
- **categories (Categorías):** category_id, name y hex_color.
- **favorites (Favoritos):** user_id y event_id.
- **images(Imágenes):** image_id y event_id.

4.3 Casos de uso

A continuación, se muestra una tabla con los casos de uso principales que definen la interacción de los actores (Organizador y Visitante) con el sistema:

- **Usuario (General):** Actor abstracto que engloba las funciones comunes de la plataforma (autenticación).
- **Visitante:** Usuario consumidor que busca mercadillos, filtra el mapa y gestiona sus favoritos.
- **Organizador:** Usuario con permisos extendidos para publicar, editar y eliminar sus propios eventos en la plataforma.

Tabla 1. Casos de uso principales del sistema.

Código	Caso de Uso	Actores Implicados	Descripción del Flujo / Objetivo
CU1	Autenticación y perfil	Usuario (General)	El usuario se registra, inicia sesión de forma segura y actualiza su fotografía de avatar desde su panel de control.
CU2	Publicación de mercadillo	Organizador	El organizador rellena el formulario de un evento, adjunta imágenes y el sistema transforma la dirección en coordenadas físicas.
CU3	Exploración en mapa	Visitante / Organizador	El usuario visualiza un mapa interactivo con pines de colores segmentados por categorías y centrados en su ubicación actual o seleccionada.
CU4	Filtrado dinámico	Visitante / Organizador	El usuario aplica filtros en la interfaz para discriminar los eventos del mapa por categorías específicas (ej. "Vintage") o por rangos de fechas.
CU5	Visualización de detalles	Visitante / Organizador	Al pulsar sobre un evento, el usuario accede a su ficha técnica completa y navega de forma interactiva por su carrusel de fotos.

CU6	Gestión de favoritos	Visitante/ Organizador	El usuario guarda un evento de su interés pulsando el icono del corazón, almacenándolo de forma persistente en su sección privada de favoritos.
CU7	Edición y eliminación manual	Organizador	El creador del evento accede a su lista de publicaciones para modificar campos de texto o dar de baja el mercadillo de forma anticipada.

5. Diseño

5.1 Diseño general / arquitectura

MEC funciona con una estructura clásica de cliente-servidor descentralizada. La aplicación se divide en tres bloques que trabajan de forma independiente:

1. **Frontend: Programado con Vue 3 y Pinia.** Se encarga de todo lo que ve el usuario, de renderizar el mapa interactivo (Leaflet.js) y de controlar los datos locales de la sesión.
2. **Backend:** Programado con Python y Flask. Es el intermediario que recibe las peticiones del Frontend, gestiona las imágenes físicamente en las carpetas y se comunica con la base de datos.
3. **Base de Datos:** Un sistema relacional en MariaDB donde se almacena de forma ordenada toda la información estructurada de la aplicación.

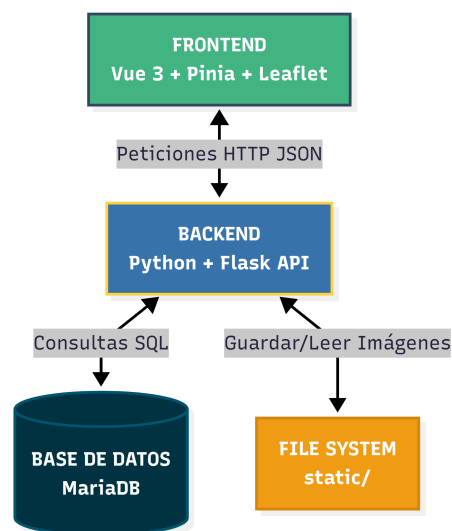


Figura 1. Diagrama simple (cajas y flechas)

5.2 Diagrama de clases (UML)

Este esquema muestra cómo se organizan los objetos reales dentro del código de la aplicación, definiendo sus atributos clave y los métodos de acceso a datos (DTO) encargados de su gestión en la base de datos:

- **Usuario:** Modela los perfiles del sistema. Alberga los datos de cuenta y ejecuta métodos de autenticación (login_user, register_user, verify_user) y gestión de perfil (update_user_info, update_user_foto, change_password).
- **Evento (Mercadillo):** Entidad central que reúne la información de cada mercadillo, incluyendo las coordenadas espaciales. Agrupa las operaciones CRUD principales (add_event, get_events, update_event_info, delete_event) y procesos automatizados (delete_past_events).
- **Categoría:** Organiza los mercadillos mediante tipos (Vintage, Solidario, etc.) y asigna el color hexadecimal del pin. Su método principal de lectura es get_categories().
- **Favorito:** Tabla relacional que vincula directamente a los usuarios con los mercadillos. Sus métodos de persistencia permiten guardar, borrar y consultar listas personalizadas (add_favorite, remove_favorite, get_favorite).

Anexo 2. Diagrama UML

5.3 Mockups / capturas de interfaz y navegación

La navegación de la aplicación se ha diseñado para que sea muy directa e intuitiva, usando el mapa interactivo como el centro de la experiencia de usuario:

Pantalla Principal: Es la portada de la web. Tiene los filtros superiores (por fechas, categorías y cercanía), el mapa interactivo con los pines de colores y las tarjetas de los mercadillos en la zona inferior.

Pantalla de Detalles: Se abre al pulsar en cualquier mercadillo. Muestra la información ampliada, la dirección exacta y el carrusel de flechas para visualizar las diferentes fotos.

Panel de Perfil y Modales: La zona privada donde el usuario puede editar su nombre, cambiar su foto de perfil y abrir el formulario para publicar un mercadillo subiendo varias imágenes a la vez en el caso del organizador, al visitante le aparecerá la opción de ir a ver sus favoritos.

Pantalla de favoritos: La zona privada donde el usuario puede ver sus eventos favoritos que ha marcado previamente.

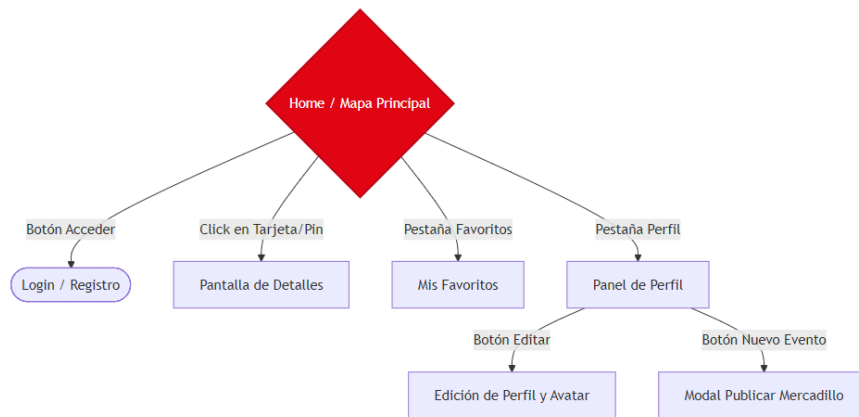


Figura 2. Mapa de pantallas

Anexo 3. Capturas de las diferentes ventanas de la app

5.4 Decisiones de diseño importantes

Durante el desarrollo del proyecto se tomaron tres decisiones técnicas fundamentales para solucionar problemas clave:

1. **Almacenamiento de múltiples fotos en un solo campo:** Para no sobre complicar la base de datos creando tablas relacionales extra solo para las fotos, se decidió guardar todas las rutas de las imágenes juntas en un único campo de texto (photo_url), separadas por comas. Posteriormente, el Frontend se encarga de procesarlas dinámicamente mediante la función `.split(',')` para generar el carrusel de imágenes de forma automática.
2. **Estructura fija para todas las respuestas (DTO):** Se implementó un estándar estricto en el Backend para enviar los datos siempre con el mismo formato (status, message, data) utilizando la función `response_wrapper`. Gracias a esto, el Frontend procesa los éxitos o los mensajes de error de forma uniforme, simplificando enormemente la captura de excepciones.
3. **Subida independiente y reactiva del avatar:** Se separó la lógica de actualización de la foto de perfil del resto de campos de texto del formulario. De este modo, la imagen se envía al servidor de forma asíncrona y se guarda en el momento exacto en que el usuario la selecciona en su dispositivo, logrando un diseño mucho más ágil, moderno y sin recargas innecesarias.

6. Implementación

6.1 Estructura del proyecto.

La aplicación está organizada separando claramente el Frontend (Vue 3) y el Backend (Flask) para que el código sea limpio y fácil de mantener. Las carpetas principales del proyecto están distribuidas así:

Frontend (/frontend):

- **src/components/**: Contiene los elementos visuales reutilizables de la interfaz, como las tarjetas de los mercadillos o los modales de los formularios.
- **src/views/**: Almacena las pantallas principales por las que navega el usuario (ej. vista principal del mapa, perfil privado, detalles del evento).
- **src/stores/**: Lógica de estado global de la aplicación utilizando Pinia.
- **src/router/**: Configuración de las rutas de navegación del frontend.
- **src/constants/**: Archivos con variables y configuraciones estáticas comunes para toda la app.
- **src/css/ y src/assets/**: Hojas de estilo y recursos estáticos (imágenes, iconos).

Backend (/backend):

- **app.py**: Punto de entrada del servidor Flask. Define los *endpoints* de la API REST y gestiona la recepción de peticiones HTTP.
- **dao.py y dto.py**: Archivos fundamentales de la arquitectura. El DAO (*Data Access Object*) interactúa con la base de datos, mientras que el DTO (*Data Transfer Object*) procesa la lógica y empaqueta las respuestas.
- **src/db.py**: Archivo de configuración que establece y maneja la conexión directa con MariaDB.
- **static/**: Directorio donde se almacenan físicamente los recursos multimedia subidos por los usuarios, divididos ordenadamente en `img_eventos/` e `img_perfil/`.

6.2 Componentes principales.

El funcionamiento interno de MEC se apoya en los siguientes pilares técnicos:

1. **Persistencia y DAOs**: La conexión a la base de datos se inicializa en `src/db.py`. Para aislar las consultas SQL del resto del código, se utiliza el patrón DAO en `dao.py`. Esto permite que todas las inserciones, lecturas y borrados en MariaDB se hagan de forma segura y centralizada mediante consultas parametrizadas.
2. **Controladores y Servicios (Patrón DTO)**: El archivo `app.py` actúa como controlador, recibiendo las peticiones del frontend y delegando el trabajo a `dto.py`. En el DTO se aplica la lógica de negocio y se estandarizan los datos usando una función `response_wrapper`. Así, el servidor siempre responde con la misma estructura predecible (`status`, `data`, `success`).
3. **UI y Cliente (Vue + Pinia)**: La interfaz gráfica está construida de forma reactiva. Pinia (`stores/`) actúa como el cerebro del lado del cliente, centralizando las peticiones al servidor y manteniendo en memoria los datos del usuario conectado o los mercadillos cargados, evitando recargas innecesarias de la página.

4. **Integración con APIs Externas:** El cliente se comunica con servicios de terceros de forma asíncrona. Se utiliza Leaflet.js para renderizar el mapa cartográfico y la API de Nominatim (OpenStreetMap) para el proceso de geocodificación, transformando el texto de la dirección del mercadillo en coordenadas exactas de latitud y longitud.

6.3 Dificultades encontradas y soluciones

Problema 1: Descontrol en la captura de errores de red.

Inicialmente, si una petición fallaba, cada componente del frontend interpretaba el error de manera distinta, provocando comportamientos inesperados en la interfaz o alertas técnicas incomprensibles para el usuario final.

- Solución: Se diseñó e implementó la función global `manageErrors` dentro del Store de Vue (Pinia) con el objetivo de unificar y procesar todos los errores del proyecto bajo un mismo criterio. Al estar sincronizada de manera nativa con el `response_wrapper` del backend, cualquier excepción o código HTTP (como errores 400 o 500) se intercepta automáticamente, traducándose de forma instantánea en un mensaje limpio, estandarizado y comprensible para el usuario.

Problema 2: Inconsistencias y fallos en la geocodificación de direcciones.

Al utilizar la API gratuita de Nominatim (OpenStreetMap) para convertir la dirección del mercadillo en coordenadas, si el organizador introducía una calle muy ambigua o sin especificar la ciudad, la API no devolvía resultados, impidiendo que el evento se posicionara en el mapa.

- Solución: Se implementó una función de limpieza y validación en el cliente (`obtenerCoordenadas`). Antes de enviar la petición a Nominatim, el código verifica si la cadena de texto contiene la ciudad y añade automáticamente parámetros de contexto (ej. ", España") para acotar la búsqueda.

Problema 3: Pérdida de reactividad en la interfaz al actualizar datos.

Tras realizar acciones como cambiar la foto de perfil o publicar un nuevo mercadillo, la interfaz visual no reflejaba los cambios inmediatamente. El usuario tenía que recargar la página del navegador manualmente (F5) para ver su nueva foto o el nuevo pin en el mapa.

- Solución: Se aprovechó el sistema de reactividad de Vue 3 y Pinia. Se configuraron las funciones asíncronas para que, al recibir un código de éxito (200 OK o 201 Created) desde el servidor, el componente emitiera un evento (`emit('actualizar')`) o volviera a llamar a la función `fetchEventos()`. Esto actualiza el estado global en memoria y fuerza al DOM a redibujar el mapa y las imágenes al instante, sin necesidad de refrescar la ventana.

7. Multiusuario concurrente

7.1 Identificación y gestión de sesiones

Para garantizar que múltiples usuarios puedan interactuar con la plataforma simultáneamente sin que sus datos se crucen, MEC utiliza un sistema de autenticación basado en credenciales (email y contraseña encriptada).

El flujo de identificación es el siguiente:

1. El usuario introduce sus credenciales en el Frontend.
2. El Backend verifica la contraseña (comprobando el hash en MariaDB) y devuelve una respuesta de éxito junto con los datos básicos del usuario (como su `id_usuario` y `tipo_usuario`).
3. El Frontend utiliza Pinia (el gestor de estado de Vue) para almacenar esta sesión en memoria del lado del cliente. Gracias a esto, la aplicación sabe en todo momento qué usuario está navegando, qué botones debe mostrarle (por ejemplo, ocultando el botón "Publicar nuevo" si es un visitante) y qué `id_usuario` debe adjuntar a las peticiones al guardar un favorito o crear un evento

7.2 Concurrencia de usuarios simultáneos

El sistema está diseñado para soportar las peticiones concurrentes de varios clientes gracias a la naturaleza del servidor Flask y la gestión de transacciones de MariaDB.

Un escenario real de uso simultáneo en la plataforma ocurre cuando:

- **Usuario A (Organizador)** está validado en la plataforma y publica un nuevo mercadillo desde su panel de control.
- **Usuario B (Visitante)** está navegando simultáneamente por el mapa de la pantalla principal. Al guardar el evento el Usuario A, MariaDB bloquea el registro sólo durante los milisegundos que dura la inserción. Inmediatamente después, si el Usuario B refresca su mapa, el nuevo marcador de color aparecerá en su pantalla sin haber interrumpido en ningún momento su navegación.

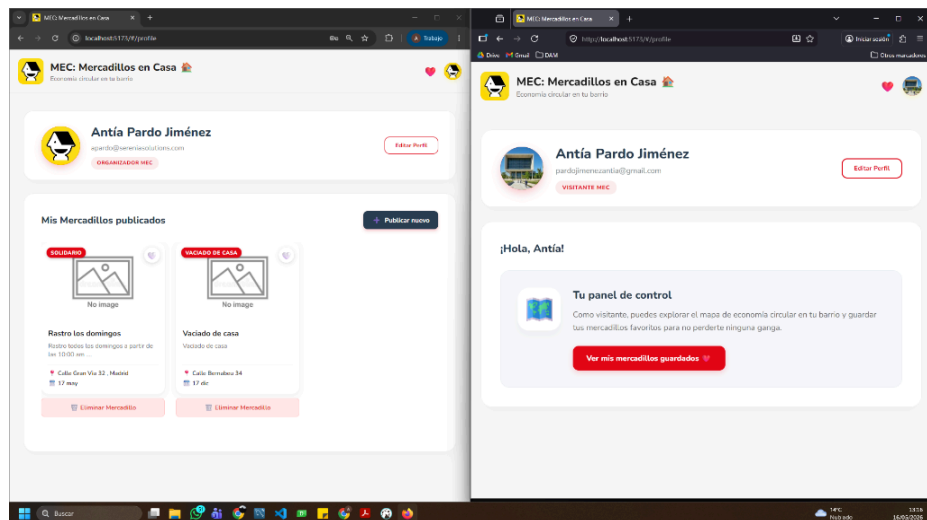


Figura 3. Captura pantalla principal y perfil demostrando concurrencia

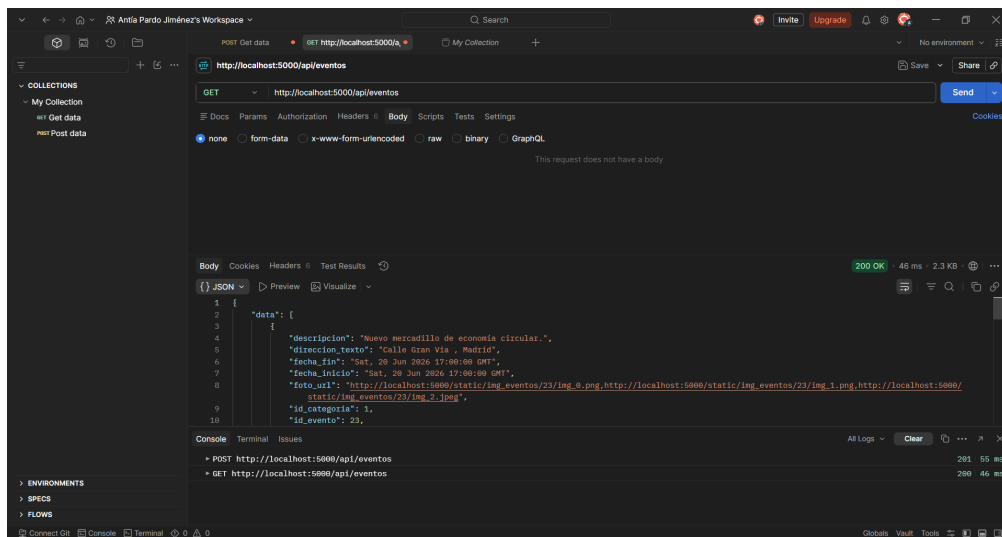


Figura 4. Demostración de peticiones concurrentes en la Consola de Postman.

7.3 Gestión de fallos de comunicación

En aplicaciones multiusuario, es habitual que se produzcan cortes de red o caídas del servidor por exceso de peticiones. MEC está blindada contra estos fallos mediante la función centralizada `manageErrors`.

Si se pierde la comunicación con el servidor (por ejemplo, si el servidor se apaga o la conexión a internet del móvil del usuario falla mientras intenta subir las fotos de un mercadillo):

1. La petición asíncrona de Axios entra en el bloque `catch`.
2. El sistema detecta que no hay respuesta del servidor o que devuelve un código 500 (Internal Server Error).
3. En lugar de bloquear la aplicación o mostrar código técnico por pantalla, el componente cancela la acción, detiene los indicadores de carga (spinners) y lanza un

mensaje amigable al usuario (ej. "Error inesperado de red" o "No se ha podido conectar con el servidor") indicándole que lo intente más tarde.

8. Despliegue e instalación

Este apartado documenta los requisitos previos, la configuración del entorno y los pasos necesarios para la puesta en marcha de la plataforma tanto en su vertiente de servidor (Backend) como de cliente (Frontend).

8.1 Manual de instalación

Requisitos previos del sistema Para poder ejecutar la plataforma de manera local, el entorno de desarrollo debe contar con el siguiente software instalado:

- Python (v3.11 o superior): Para la ejecución del servidor Backend.
- Node.js y NPM (v10.8+): Para la gestión de dependencias y compilación del Frontend.
- MariaDB (v11.4+): Servidor de base de datos relacional activo en el puerto por defecto (3306).
- Git: Para el control de versiones y clonación del repositorio.

Pasos para el arranque del Backend (Flask)

1. Clonar el repositorio del proyecto en el directorio local.
2. Abrir una terminal en la carpeta raíz del backend.
3. Crear y activar un entorno virtual para aislar las dependencias:
 - a. En consola general: `python -m venv venv`
 - b. En Windows: `venv\Scripts\activate`
 - c. En macOS/Linux: `source venv/bin/activate`
4. Instalar las dependencias listadas en el fichero de configuración:
 - a. Comando: `pip install -r requirements.txt`
5. Iniciar el servidor de desarrollo:
 - a. Comando: `python app.py`

Pasos para el arranque del Frontend (Vue 3 + Vite)

1. Abrir una nueva terminal y navegar hasta el directorio del frontend.
2. Instalar los paquetes y dependencias de Node:
 - a. Comando: `npm install`

3. Levantar el servidor de desarrollo de Vite:
 - a. Comando: `npm run dev`
4. La aplicación estará accesible desde el navegador en `http://localhost:5173`.

8.2 Configuración

Para garantizar la seguridad y portabilidad del código, las credenciales y variables de entorno no se incluyen en el repositorio. Es necesario crear archivos `.env` tanto en el directorio del frontend como en el del backend basándose en los ficheros de ejemplo proporcionados (`.env.example`).

Variables del Backend (.env) Este archivo gestiona la conexión a la base de datos y la seguridad de la API:

- `FLASK_APP=app.py`
- `FLASK_ENV=development`
- `SECRET_KEY=clave_secreta`
- `DB_HOST=localhost`
- `DB_USER=root`
- `DB_PASSWORD= contraseña`
- `DB_NAME= nombre_base_datos_mec`

Variables del Frontend (.env) Este archivo indica al cliente dónde debe realizar las peticiones asíncronas: `VITE_API_URL=http://192.168.1.XX:5000/api`, donde 192.168.1.XX corresponde a la dirección IP local del servidor que aloja la API, permitiendo la conexión tanto desde navegadores web como desde emuladores móviles

8.3 Datos de ejemplo

Para facilitar las pruebas y la evaluación del Producto Mínimo Viable (MVP), el proyecto incluye un script de inicialización que construye la estructura relacional e inserta un conjunto de datos ficticios (usuarios organizadores, visitantes, categorías y mercadillos de prueba).

Procedimiento de carga (Importación SQL):

1. Acceder al gestor de bases de datos (por ejemplo, a través de consola o una interfaz gráfica como DBeaver).
2. Crear una base de datos vacía con el nombre especificado en el archivo `.env`.
3. Ejecutar el script SQL de volcado ubicado en la carpeta `database/` del proyecto para generar las tablas y los datos de prueba. Comando de ejemplo (si se usa la consola):
`SOURCE ruta/al/archivo/init_db.sql;`

Una vez ejecutado, el sistema contará con credenciales de prueba preconfiguradas para iniciar sesión sin necesidad de realizar nuevos registros.

8.4 Manual de usuario

A continuación, se detalla el funcionamiento básico de la aplicación MEC (Mercadillos en Casa) desde la perspectiva del usuario, ilustrando las pantallas y flujos principales.

1. Acceso y Registro Para interactuar con la plataforma, el usuario debe disponer de una cuenta.

- **Iniciar sesión:** Los usuarios registrados pueden acceder introduciendo su correo electrónico y contraseña.
- **Crear cuenta:** Los nuevos usuarios deben rellenar un formulario con sus datos básicos y seleccionar su rol en la plataforma: "Quiero visitar mercadillos" (Visitante) o "Quiero organizar mercadillos" (Organizador).

2. Pantalla Principal (Exploración) Una vez dentro, la pantalla principal ofrece un mapa interactivo y un listado de los mercadillos disponibles.

- **Filtros de búsqueda:** Permiten acotar los resultados por categoría de evento, fechas específicas o proximidad mediante el botón **Cerca de mí**.
- **Tarjetas de evento:** Muestran un resumen rápido con el título, ubicación y fecha del mercadillo.

3. Detalles del Evento Al hacer clic en cualquier mercadillo de la pantalla principal, el usuario accede a la vista detallada. Aquí se muestra toda la información completa proporcionada por el organizador, incluyendo la descripción, un carrusel de imágenes, fecha exacta y ubicación. Además, incluye un botón directo para **Añadir a favoritos**.

4. Mis Favoritos Sección dedicada exclusivamente a los usuarios que desean guardar eventos para no perderlos de vista. Muestra un listado limpio con las tarjetas de los mercadillos que el usuario ha marcado previamente con el icono del corazón.

5. Panel de Control (Perfiles de Usuario) La aplicación adapta su interfaz dependiendo del rol elegido por el usuario durante el registro:

- **Perfil Visitante:** Muestra un panel de control simplificado con un acceso directo a los mercadillos guardados.
- **Perfil Organizador:** Actúa como centro de gestión. Permite visualizar los mercadillos publicados por el propio usuario, editar sus datos, eliminarlos si se cancelan, o utilizar el botón **Publicar nuevo** para crear un evento desde cero.

6. Configuración de la Cuenta Desde el panel de control, cualquier usuario puede acceder al apartado de Editar Perfil. En esta pantalla se permite actualizar la información personal (como el nombre o la foto de perfil) y cuenta con un formulario de seguridad dedicado exclusivamente al cambio de contraseña.

9. Pruebas

9.1 Plan de pruebas

¿Qué se va a probar? El plan de pruebas se centra en validar el correcto funcionamiento del Producto Mínimo Viable (MVP) en sus tres capas principales:

- Interfaz de Usuario (Frontend): Respuesta de los componentes reactivos de Vue, adaptabilidad del diseño (Responsive Design) en dispositivos móviles y de escritorio, e interactividad del mapa de Leaflet.
- Lógica de Negocio e Integración (Conexión API): Correcto flujo de datos entre el cliente y el servidor a través de Axios, validando que el manejador global de errores procese adecuadamente las excepciones de red.
- Persistencia y Backend (Base de datos y Flask): Integridad referencial en MariaDB, correcta inserción y borrado de registros, y el manejo de carga de archivos (imágenes) al servidor.

¿Con qué objetivo? El objetivo principal es asegurar la estabilidad, seguridad y usabilidad de la plataforma antes de su despliegue final. Se busca garantizar que los flujos críticos (autenticación, creación de mercadillos, visualización en el mapa y gestión de favoritos) funcionen sin interrupciones, y que el sistema sea capaz de recuperarse o informar adecuadamente al usuario ante entradas de datos no válidas o fallos de conectividad.

Tabla 2. Pruebas principales del sistema.

Descripción del Test	Precondiciones	Pasos a seguir	Resultado Esperado
Registro exitoso de nuevo Organizador	No estar logueado en el sistema.	1. Ir a "Registrarse". 2. Rellenar datos válidos. 3. Elegir rol "Quiero organizar". 4. Pulsar "Registrarme".	El sistema crea el usuario, inicia sesión automáticamente y redirige al panel de Organizador.
Bloqueo por credenciales incorrectas	Tener un usuario ya registrado en la BD.	1. Ir a "Iniciar Sesión". 2. Introducir un email válido pero una contraseña falsa. 3. Pulsar "Entrar".	El sistema deniega el acceso y muestra un mensaje de error indicando credenciales inválidas.
Filtrado de mercadillos por fecha	Existir al menos 2 eventos en distintas fechas.	1. Ir a la pantalla principal. 2. Seleccionar un rango de fechas en el buscador superior.	El mapa y la lista se actualizan mostrando solo los mercadillos dentro de esas fechas.

Añadir evento a Favoritos	Iniciar sesión como "Visitante".	1. Abrir los detalles de un evento. 2. Pulsar el botón de "Añadir a favoritos". 3. Ir a la pestaña "Favoritos".	El icono cambia visualmente y el mercadillo aparece listado en la sección "Mis Favoritos".
Publicar un nuevo mercadillo	Iniciar sesión como "Organizador".	1. Ir al Perfil. 2. Pulsar "Publicar nuevo". 3. Rellenar formulario y ubicación. 4. Guardar.	El mercadillo se guarda en la base de datos y aparece inmediatamente en el panel del organizador.
Edición de un mercadillo existente	Tener un mercadillo propio publicado.	1. Ir al Perfil. 2. Pulsar "Editar Mercadillo". 3. Cambiar el título. 4. Guardar cambios.	El sistema actualiza los datos y el nuevo título se refleja en los detalles del evento.
Eliminación de un mercadillo	Tener un mercadillo propio publicado.	1. Ir al Perfil. 2. Pulsar "Eliminar Mercadillo". 3. Confirmar en el modal de advertencia.	El evento desaparece de la base de datos, del panel del usuario y del mapa público.
Cambio de contraseña seguro	Estar logueado en el sistema.	1. Ir a "Editar Perfil". 2. Escribir contraseña actual y una nueva. 3. Confirmar cambio.	El sistema valida la contraseña antigua, actualiza a la nueva y muestra un mensaje de éxito.

El resultado obtenido de todas las pruebas ha sido exitoso.

Anexo 4. Capturas de éxito de los casos de prueba

10. Resultados, conclusiones y vías futuras

10.1 Objetivos alcanzados

En las etapas iniciales del proyecto se establecieron diversos objetivos técnicos y funcionales con el propósito de desarrollar un Producto Mínimo Viable (MVP) sólido y escalable. A continuación, se detalla el grado de consecución de dichas metas:

Objetivos cumplidos:

- **Desarrollo Full Stack del MVP:** Se ha implementado con éxito una arquitectura cliente-servidor completamente funcional utilizando Vue 3 y Flask, garantizando la persistencia de datos mediante MariaDB.

- **Gestión de roles y autenticación:** Se ha logrado separar correctamente la lógica y las interfaces entre los perfiles de "Organizador" y "Visitante", securizando el acceso a las rutas privadas.
- **Integración de geolocalización interactiva:** Se ha cumplido el objetivo de integrar Leaflet y la API de Nominatim, logrando que la introducción de una dirección en texto se transforme de manera transparente en coordenadas renderizadas en el mapa interactivo.
- **Manejo robusto de excepciones:** Gracias a la implementación de la función global de gestión de errores, la plataforma es tolerante a fallos de red y caídas del servidor, protegiendo la experiencia del usuario.

Objetivos no cumplidos o parcialmente cumplidos:

- **Autenticación con redes sociales (Social Login):** Originalmente se planteó permitir el registro mediante cuentas de Google o Facebook usando OAuth2. No obstante, para mantener el control total sobre la base de datos de usuarios en esta primera versión del producto, se decidió priorizar e implementar un sistema de autenticación propio y seguro mediante tokens JWT.
- **Recuperación de contraseña por correo electrónico:** Actualmente el sistema de seguridad cubre el registro y el inicio de sesión, pero el flujo automatizado de "He olvidado mi contraseña" con envío de tokens por SMTP ha quedado relegado a una segunda fase por limitaciones de tiempo en el calendario académico.

10.2 Conclusiones

Aprendizaje técnico y personal

El desarrollo de esta plataforma ha supuesto un desafío integral que ha permitido consolidar los conocimientos adquiridos a lo largo de la formación. Ha requerido afrontar problemas reales de ingeniería de software, como la sincronización de estados globales de forma reactiva, el tratamiento de datos espaciales (coordenadas) en el frontend y el diseño de una API REST segura. Además, la necesidad de centralizar el manejo de errores ha fomentado la adopción de buenas prácticas de arquitectura y código limpio, elevando la calidad técnica y la mantenibilidad del producto.

Aportación del proyecto

A nivel de negocio y utilidad social, este proyecto aporta la base tecnológica necesaria para digitalizar y centralizar un sector tradicionalmente analógico e informal. La aplicación ofrece una solución directa al nicho de mercados de segunda mano y la economía circular en España, facilitando que los usuarios encuentren opciones de consumo responsable en su propio barrio.

10.3 Vías futuras / mejoras

De cara a la evolución de la plataforma más allá de este Trabajo de Fin de Ciclo, se plantean las siguientes líneas de mejora realistas para escalar el producto:

- **Sistema de notificaciones *Push* y Alertas:** Implementar avisos en el dispositivo del visitante (o vía correo electrónico) cuando se publique un nuevo evento cerca de su ubicación o se actualice información de un mercadillo guardado en sus favoritos.
- **Sistema de valoraciones y reseñas:** Desarrollar un mecanismo donde los visitantes puedan puntuar del 1 al 5 y dejar comentarios sobre los mercadillos a los que han asistido, fomentando la confianza y creando una comunidad de organizadores verificados.
- **Chat directo Organizador-Visitante:** Integrar WebSockets para habilitar la comunicación en tiempo real, permitiendo a los interesados preguntar por la disponibilidad de artículos concretos antes de desplazarse físicamente al lugar del evento.
- **Panel de analíticas para organizadores:** Crear una vista de estadísticas donde el creador del evento pueda ver cuántos usuarios han visualizado su mercadillo o cuántos lo han añadido a favoritos, aportándole valor añadido sobre el impacto de su publicación.

11. Bibliografía / Webgrafía

A continuación se detallan las fuentes documentales, documentaciones oficiales, librerías y recursos externos consultados y utilizados para el desarrollo de esta plataforma.

Documentación Oficial (Frontend)

- Vue.js (v3): Documentación oficial del framework y Composition API.
 - Recuperado de: <https://vuejs.org/guide/introduction.html>
- Pinia: Documentación oficial del gestor de estado para Vue.
 - Recuperado de: <https://pinia.vuejs.org/>

Documentación Oficial (Backend y Base de Datos)

- Python (v3.11): Documentación oficial del lenguaje de programación.
 - Recuperado de: <https://docs.python.org/3/>
- Flask (v3.0): Documentación oficial del microframework web para Python.
 - Recuperado de: <https://flask.palletsprojects.com/>
- MariaDB: Base de conocimientos y documentación técnica oficial de MariaDB.
 - Recuperado de: <https://mariadb.com/kb/en/documentation/>

APIs, Librerías y Dependencias

- Leaflet.js: Librería de JavaScript de código abierto para mapas interactivos *mobile-friendly*.
 - Recuperado de: <https://leafletjs.com/reference.html>

Herramientas de Desarrollo y Testing

- Visual Studio Code: Documentación del entorno de desarrollo.
 - Recuperado de: <https://code.visualstudio.com/docs>
- Postman: Herramienta de testing y desarrollo de APIs.
 - Recuperado de: <https://learning.postman.com/docs/introduction/overview/>
- Git: Documentación oficial del sistema de control de versiones.
 - Recuperado de: <https://git-scm.com/doc>